Chapter 2. Tokens and Embeddings

Tokenization and Embedding

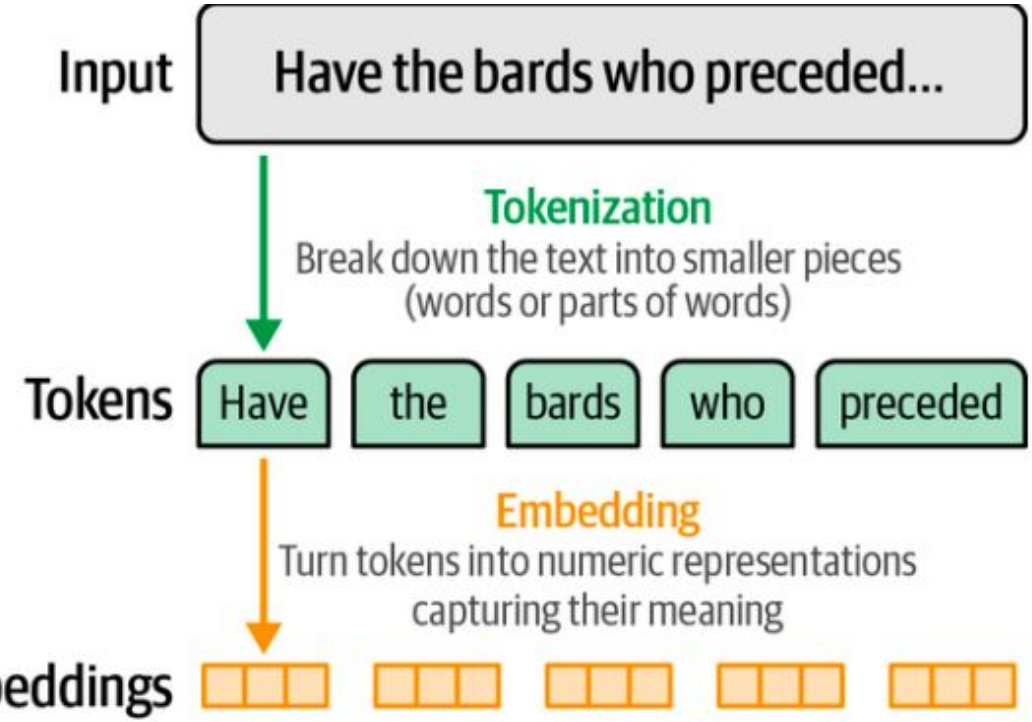


Figure 2-1. Language models deal with text in small chunks called tokens. For the language model to compute language, it needs to turn tokens into numeric representations called embeddings.

Loading and Running an LLM

```
from transformers import AutoModelForCausalLM, AutoTokenizer

# Load model and tokenizer
model = AutoModelForCausalLM.from_pretrained(
    "microsoft/Phi-3-mini-4k-instruct",
    device_map="cuda",
    torch_dtype="auto",
    trust_remote_code=True,
)
tokenizer = AutoTokenizer.from_pretrained("microsoft/Phi-3-mini-
4k-instruct")
```

We can then proceed to the actual generation. We first declare our prompt, then tokenize it, then pass those tokens to the model, which generates its output. In this case, we're asking the model to only generate 20 new tokens:

```
prompt = "Write an email apologizing to Sarah for the tragic
gardening mishap. Explain how it happened.<|assistant|>"

# Tokenize the input prompt
input_ids = tokenizer(prompt,
return_tensors="pt").input_ids.to("cuda")

# Generate the text
generation_output = model.generate(
  input_ids=input_ids,
  max_new_tokens=20
)

# Print the output
print(tokenizer.decode(generation_output[0]))
```

Output:

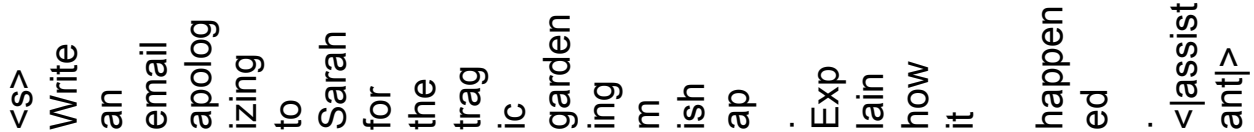
```
<s> Write an email apologizing to Sarah for the tragic
gardening mishap. Explain how it happened.<|assistant|> Subject:
My Sincere Apologies for the Gardening Mishap
```

We first declare our prompt, then tokenize it, then pass those tokens to the model, which generates its output. In this case, we're asking the model to only generate 20 new tokens:

Input to LLM is the Unique TokenID

Let’s print input_ids to see what it holds inside:
tensor([[1, 14350, 385, 4876, 27746, 5281, 304, 19235, 363, 278, 25305, 293, 16423, 292, 286, 728, 481, 29889, 12027, 7420, 920, 372, 9559, 29889, 32001]], device='cuda:0')

This reveals the inputs that LLMs respond to, a series of integers as shown. Each one is the unique ID for a specific token (character,word, or part of a word). These IDs reference a table inside the tokenizer containing all the tokens it knows.



Output of LLM are Tokens

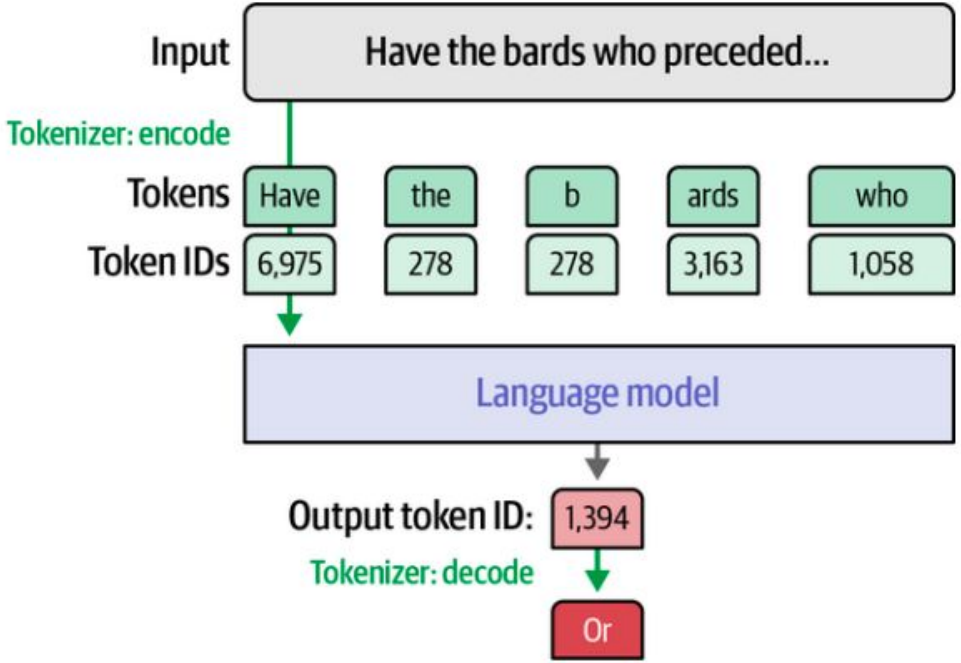


Figure 2-5. Tokenizers are also used to process the output of the model by converting the output token ID into the word or token associated with that ID.

In addition to being used to process the input text into a language model, tokenizers are used on the output of the language model to turn the resulting token ID into the output word or token associated with it.

Properties of Tokens

This is how the tokenizer broke down our input prompt. Notice the following:

- The first token is ID 1 (<s>), a special token indicating the beginning of the text.
- Some tokens are complete words (e.g., Write, an, email).
- Some tokens are parts of words (e.g., apolog, izing, trag, ic).
- Punctuation characters are their own token.
- The space character does not have its own token.

Input to LLM is the Unique TokenID

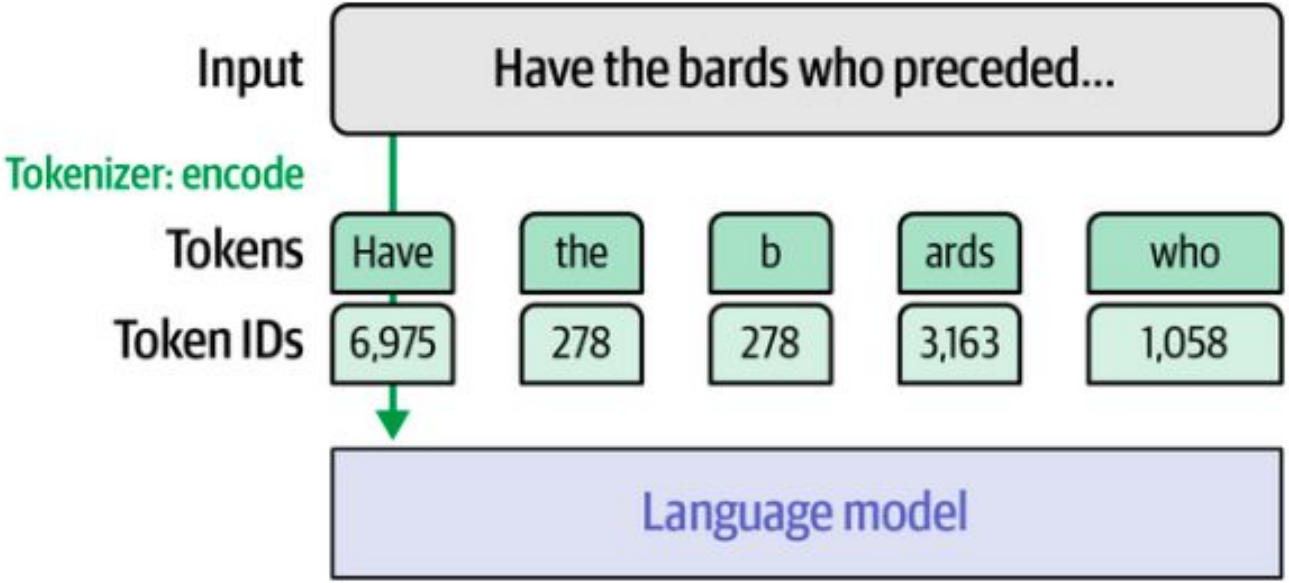


Figure 2-4. A tokenizer processes the input prompt and prepares the actual input into the language model: a list of token IDs. The specific token IDs in the figure are just demonstrative.

What Are Tokens?

Definition: Smallest units of text a model processes
Examples: Words, subwords, characters
Tokenization strategies: whitespace, byte-pair encoding (BPE), WordPiece

Why Tokens?

Language models don’t understand text the way humans do. They need **numerical input** — and raw text is too ambiguous, too varied, and too complex to feed directly into a model.

Tokenization is the bridge between human language and machine-readable format

Tokenization Methods

- Word
- Subword
- Character
- Byte

Word Level Tokenization

- It’s the process of **splitting a sentence or text into individual words**, treating each word as a separate token. These tokens are the basic units that NLP models use to understand and process language.
- **Simplifies text** for downstream tasks like classification, translation, or sentiment analysis.
- **Preserves semantic units** better than character-level tokenization.
- **Works well for languages with clear word boundaries**, like English

Input: "Machine learning is fascinating."
Output: ["Machine", "learning", "is", "fascinating"]

Disadvantages Word Level Tokenization

- One challenge with word tokenization is that the tokenizer may be **unable to deal with new words** that enter the dataset after the tokenizer was trained.
- Also results in a vocabulary that has a lot of tokens with minimal differences between them (e.g., apology, apologize, apologetic, apologist). This latter challenge is resolved by subword tokenization as it has a token for apolog, and then suffix tokens (e.g., -y,-ize, -etic, -ist)

Subword Level Tokenization

Subword-level tokenization is a clever middle ground between word-level and character-level tokenization. It breaks words into smaller, meaningful units—called **subwords**—which helps models handle rare words, misspellings, and morphological variants more gracefully.

Word: "unhappiness"
Subword tokens: ["un", "happi", "ness"]

Advantages

- **Handles out-of-vocabulary (OOV) words** by breaking them into known subwords.
- **Reduces vocabulary size** while maintaining semantic richness.
- **Improves generalization** across word forms (e.g., “run”, “running”, “runner”).

Disadvantages

- May produce **non-intuitive splits** (e.g., “##ing” or “@@ness”).
- Requires **pre-training** to build the subword vocabulary.
- Slightly more complex to implement than word-level tokenization.

Character Level Tokenization

Character-level tokenization is the process of breaking down text into **individual characters**, including letters, digits, punctuation, and even spaces. It's the most granular form of tokenization and can be surprisingly powerful in certain NLP tasks.

Input: "Token"
Output: ["T", "o", "k", "e", "n"]

Advantages

Character-level tokenization is especially useful when:

- **Spelling errors or typos** are common (e.g., user-generated content)
- **Languages lack clear word boundaries** (e.g., Chinese, Thai)
- **Creative or noisy text** is involved (e.g., social media, poetry)
- You want to **build models from scratch** without relying on predefined vocabularies
- **No out-of-vocabulary (OOV) issues**: Every character is known.
- **Language-agnostic**: Works across scripts and writing systems.
- **Fine-grained control**: Useful for tasks like spelling correction, name normalization, or stylometry.
-

Limitations

- **Longer sequences**: Text becomes much longer, increasing computational cost.
- **Loss of semantic units**: Characters alone don't carry meaning like words or subwords do.
- **Harder to model context**: Requires deeper architectures to capture long-range dependencies.

Byte-Level Tokenization

Byte tokenization method breaks down tokens into the individual bytes that are used to represent unicode characters.

It treats **text as a sequence of raw bytes**, rather than characters or words, allowing for **universal coverage** across languages, symbols, and even emojis.

Input: "café"
UTF-8 bytes: [99, 97, 102, 195, 169]
Tokens: ["c", "a", "f", "Ã", "©"]

Advantages

- **No unknown tokens**: Every character, emoji, or symbol can be represented as bytes.
- **Compact base vocabulary**: Just 256 tokens (one for each byte).
- **Language-agnostic**: Works across all scripts and writing systems.
- **Handles noisy or creative text**: Great for social media, code, or multilingual input.

Tokenization Methods

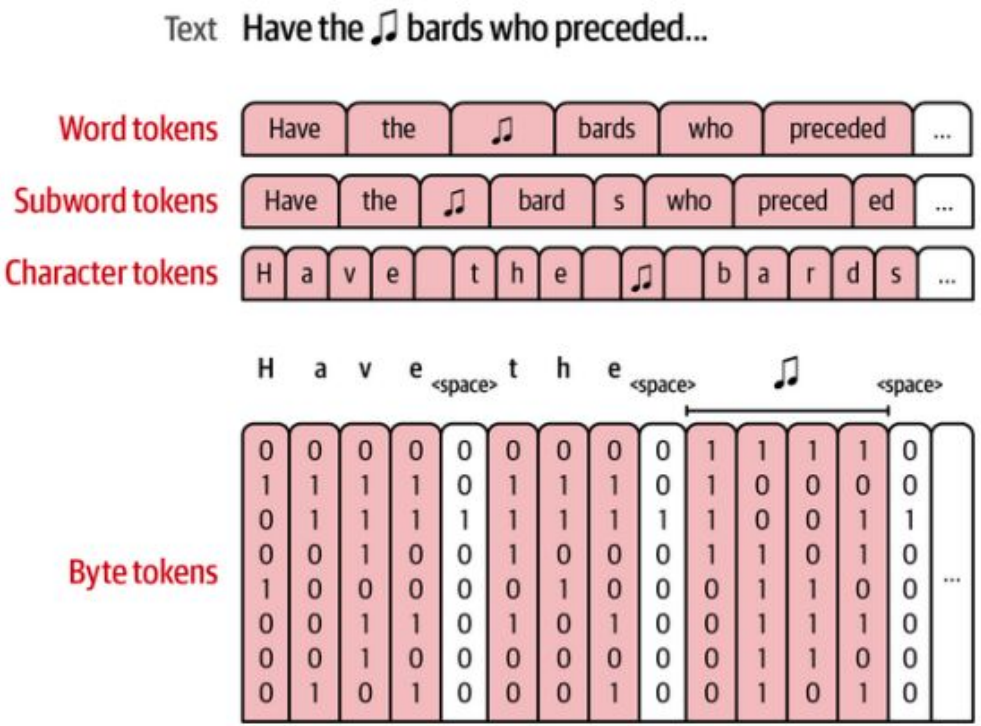


Figure 2-6. There are multiple methods of tokenization that break down the text to different sizes of components (words, subwords, characters, and bytes).

Problems Solved by Tokenization

Challenge	How Tokenization Helps
Words vary in form (run, running, ran)	Breaks into subwords → captures shared meaning
Rare or new words (photosynthesis)	Splits into known chunks → avoids unknown tokens
Misspellings or typos	Subword units still make sense (transfrmers)
Multilingual text	Uses shared vocabulary across languages
Model input size limits	Controls token count → fits within model window

Subword Level Tokenization Methods

Feature	Byte-Pair Encoding (BPE)	WordPiece	SentencePiece
Core Idea	Greedy merging of frequent character pairs	Likelihood-based subword merges	Unigram LM or BPE over raw text
Training Input	Pre-tokenized text	Pre-tokenized text	Raw text (no preprocessing needed)
Lossiness	Fully lossless (preserves spaces)	Lossy (spaces not preserved)	Partially lossless (preserves one space)
Sampling Capability	Deterministic	Deterministic	Can sample multiple tokenizations
Multilingual Support	Limited	Limited	Strong (used in multilingual models)
Vocabulary Construction	Frequency-based merges	Likelihood-based merges	Probabilistic model over subwords
Used In LLMs	GPT-2, GPT-3, GPT-4, Falcon, RoBERTa	BERT, DistilBERT, ALBERT	T5, XLNet, mT5, ByT5, PaLM

1. Byte Pair Encoding

1 Corpus of words, Number of merge operations N Tokenized corpus, Subword vocabulary
2 **Preprocess: for** *each word in corpus* **do**
3 **end**
4 Split word into characters Append end-of-word marker (e.g., _)
5 Initialize vocabulary with all unique characters and _
6 **for** *merge_step* $= 1$ *to* N **do**
7 **end**
8 Count frequency of all adjacent symbol pairs in corpus Identify most frequent pair (a, b) Merge (a, b) into new symbol ab Replace all occurrences of (a, b) with ab in corpus Add ab to vocabulary
9 **return** Final tokenized corpus and vocabulary

BPE - Example

Byte Pair Encoding Example: `aa abc abc`

Initial Corpus: `["aa", "abc", "abc"]`

Step 1: Preprocessing

- "aa" → `["a", "a", "_"]`
- "abc" → `["a", "b", "c", "_"]`
- "abc" → `["a", "b", "c", "_"]`

Step 2: Iterative Merging

1. **Iteration 1:**
   - Most frequent pair: `("a", "b")`
   - Merge into: `"ab"`
   - Corpus updates:
     - `["a", "a", "_"]`
     - `["ab", "c", "_"]`
     - `["ab", "c", "_"]`
   - Vocabulary: `["a", "b", "c", "_", "ab"]`
2. **Iteration 2:**
   - Most frequent pair: `("ab", "c")`
   - Merge into: `"abc"`
   - Corpus updates:
     - `["a", "a", "_"]`
     - `["abc", "_"]`
     - `["abc", "_"]`
   - Vocabulary: `["a", "b", "c", "_", "ab", "abc"]`

3. **Iteration 3:**
    - Most frequent pair: `("a", "a")`
    - Merge into: `"aa"`
    - Corpus updates:
        - `["aa", "_"]`
        - `["abc", "_"]`
        - `["abc", "_"]`
    - Vocabulary: `["a", "b", "c", "_", "ab", "abc", "aa"]`

Final Tokenized Corpus:

- `["aa", "_"]`
- `["abc", "_"]`
- `["abc", "_"]`

Final Vocabulary: `["a", "b", "c", "_", "ab", "abc", "aa"]`

Homework BPE

Imagine a Corpus of the following words mentioned along with their frequencies meaning `"hug"` was present 10 times in the corpus, `"pug"` 5 times, `"pun"` 12 times, `"bun"` 4 times, and `"hugs"` 5 times.

("hug", 10), ("pug", 5), ("pun", 12), ("bun", 4), ("hugs", 5)

Using BPE algorithm iterate for 5 merge steps and return the final vocabulary as well as the final tokenized corpus.

Hint: In the first iteration the pair (u,g) is present (10+5+5=) 20 times, (p,u) is present 17 times and (u,n) is present 16 times.

Wordpiece - Training Algorithm

- WordPiece is the tokenization algorithm Google developed to pretrain BERT. It has since been reused in quite a few Transformer models based on BERT, such as DistilBERT, MobileBERT, Funnel Transformers, and MPNET. It's very similar to BPE in terms of the training, but the actual tokenization is done differently.
- Then, again like BPE, WordPiece learns merge rules. The main difference is the way the pair to be merged is selected. Instead of selecting the most frequent pair, WordPiece computes a score for each pair, using the following formula:

$$score=(freq_of_pair) / (freq_of_first_element \times freq_of_second_element)$$

- By dividing the frequency of the pair by the product of the frequencies of each of its parts, the algorithm prioritizes the merging of pairs where the individual parts are less frequent in the vocabulary. For instance, it won't necessarily merge ("un", "##able") even if that pair occurs very frequently in the vocabulary, because the two pairs "un" and "##able" will likely each appear in a lot of other words and have a high frequency. In contrast, a pair like ("hu", "##gging") will probably be merged faster (assuming the word "hugging" appears often in the vocabulary) since "hu" and "##gging" are likely to be less frequent individually.

Wordpiece- Tokenization Algorithm

WordPiece Tokenization with Vocabulary: `[UNK, a, aa, ab, abc, ##b, ##c]`

Corpus: `["aa", "abc", "abc"]`

Case 1: `abc` and `ab` are in the vocabulary

- `"aa"`:
  - Longest prefix match: `"aa"` ∈ vocab
  - Tokenized: `["aa"]`
- `"abc"`:
  - Longest prefix match: `"abc"` ∈ vocab
  - Tokenized: `["abc"]`
- Final Tokenized Corpus: `[["aa"], ["abc"], ["abc"]]`

Case 2: `abc` and `ab` are not in the vocabulary
Step-by-step Tokenization of "abc":

1. Start at position 0: Try longest match from vocab
   - Try "abc" → not in vocab
   - Try "ab" → not in vocab
   - Try "a" → match
   - Emit: `"a"`
2. Position 1: Remaining string is "bc"
   - Try "##bc" → not in vocab
   - Try "##b" → match
   - Emit: `"b"`
3. Position 2: Remaining string is "c"
   - Try "##c" → match
   - Emit: `"c"`

Final Tokenization: `["a", "##b", "##c"]`
Note: WordPiece does **not** backtrack or look ahead to match `"##bc"` unless it's explicitly in the vocabulary. It only checks longest valid prefix at each position. **Fallback Behavior:**

- If no prefix match is found at any step, use `[UNK]`.
- Example: `"xyz"` → `["[UNK]"]`

How WordPiece Tokenization Works Post-Training

Once the vocabulary is trained (typically using likelihood maximization over a corpus), the tokenization process follows this **greedy strategy**:

1. **Start at the beginning of the word.**
2. **Find the longest substring** that matches a token in the vocabulary.
3. **Add that token to the output.**
4. **Move forward** in the word and repeat until the entire word is tokenized.

Let's say the WordPiece vocabulary includes: `["un", "##aff", "##able", "##ability"]`

Tokenizing `"unaffable"` would proceed as:

- Start with `"unaffable"`
- Match `"un"` → ✅
- Remaining: `"affable"`
- Match `"##aff"` → ✅
- Remaining: `"able"`
- Match `"##able"` → ✅
- Final tokens: `["un", "##aff", "##able"]`

SentencePiece

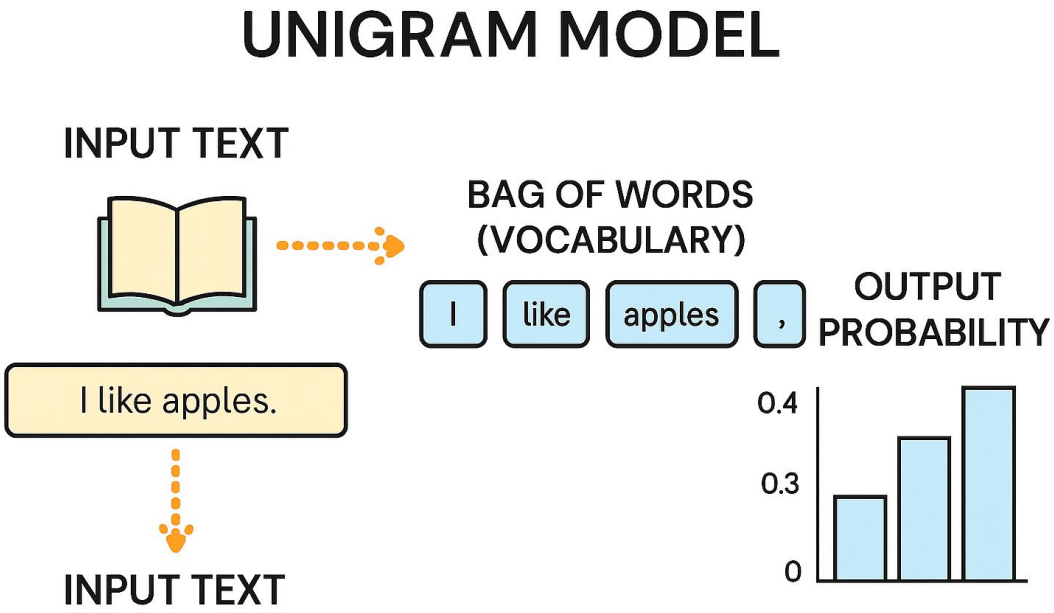
Overview

- SentencePiece is a language-independent subword tokenization algorithm developed by Google, de-signed to train directly on raw text without requiring pre-tokenization. Unlike WordPiece or BPE, which assume space-separated words as input, SentencePiece treats the input as a raw stream of Unicode characters and learns subword units using either Byte Pair Encoding (BPE) or Unigram Language Model.
- This approach enables consistent handling of whitespace, punctuation, and multilingual corpora. SentencePiece introduces a special meta-symbol to represent spaces, allowing
- it to model word boundaries explicitly.
- The Unigram Language Model is one of the core algorithms used in SentencePiece for subword tokenization. Unlike BPE, which builds the vocabulary by merging frequent pairs, the Unigram approach starts with a large initial vocabulary of candidate subwords and iteratively prunes it
- At each step, the algorithm evaluates the likelihood of the corpus given the current vocabulary and removes the subwords whose removal least increases the overall loss.
- The most used model for sentencepiece tokenization is the Unigram Model

Unigram Model

- The Unigram algorithm is used in combination with SentencePiece, which is the tokenization algorithm used by models like AlBERT, T5, mBART, Big Bird, and XLNet.

- The **Viterbi algorithm** is used to find the **most probable segmentation**—i.e., the sequence of tokens that maximizes the product of their unigram probabilities.



Viterbi Decoding in the Unigram Model: Example with ”abc

$$\text{Cost}(t_i) = -\log p_i$$

Vocabulary and Costs

We assume the following vocabulary and probabilities:

Token	**Probability** p	**Cost** $-\log p$
a	0.20	1.61
b	0.20	1.61
c	0.20	1.61
ab	0.15	1.90
bc	0.15	1.90
abc	0.10	2.30

Possible Segmentations of the word “abc”

Optimal Path via Viterbi Decoding
The Viterbi algorithm selects the path with the minimum total cost. In this case:

Best Segmentation = ["abc"], Total Cost = 2.30

This segmentation is the most probable under the Unigram model.

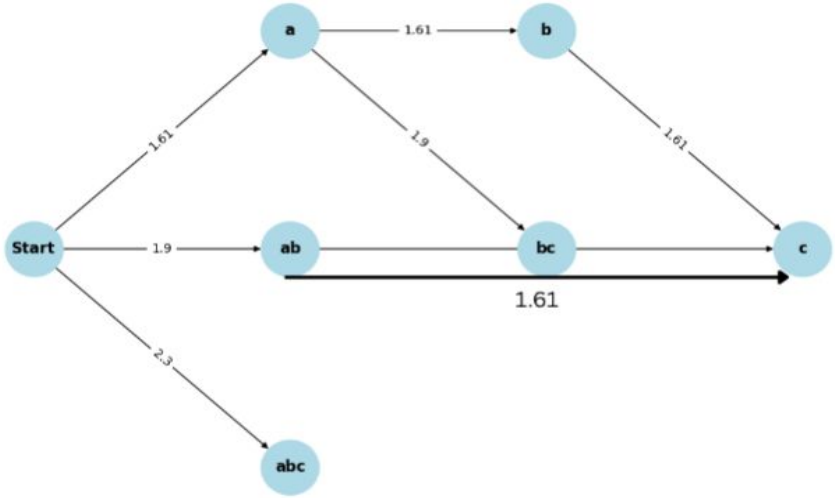


Figure 4: The cost lattice for ”abc”

- Segmentation: `["a", "b", "c"]`
  Cost: $1.61 + 1.61 + 1.61 = 4.83$
- Segmentation: `["ab", "c"]`
  Cost: $1.90 + 1.61 = 3.51$
- Segmentation: `["a", "bc"]`
  Cost: $1.61 + 1.90 = 3.51$
- Segmentation: `["abc"]`
  Cost: 2.30

Learned probabilities and the EM Algorithm

Learned Probabilities

The probability p(t) for a subword is not just the raw frequency in the corpus but rather a learned probability that reflects how often the subword appears in the valid segmentations of the corpus during training.

The Unigram model learns the probability of each subword by:

- Starts with a probability of subwords
- Iteratively adjusts the probability of each subword based on how likely it appears in valid segmentation of the training corpus.

Unigram Model uses the EM algorithm that iteratively refines token probabilities to maximize the likelihood of the corpus.

- E-step: Compute expected counts for each token based on all valid segmentations.
- M-step: Update token probabilities using normalized expected counts.

Example - Learned Probailities.

Example Corpus: aa abc abc

Token	Probability
a	0.1
aa	0.3
abc	0.6

Initial Vocabulary and Probabilities

Step -1 Find all the valid segmentations of the Corpus

🧮 Final Valid Segmentations of Entire Corpus

We now combine valid segmentations of each part:

1. `[aa] [abc] [abc]`
2. `[a, a] [abc] [abc]`

These are the **only valid segmentations** using the given vocabulary.

Step-2 Find the Total Probability for Each segmentation

🔹 Segmentation 1: `[aa] [abc] [abc]`

- Probability:
  $P = 0.3 \times 0.6 \times 0.6 = 0.108$P = 0.3 \times 0.6 \times 0.6 = 0.108

🔹 Segmentation 2: `[a, a] [abc] [abc]`

- Probability:
  $P = 0.1 \times 0.1 \times 0.6 \times 0.6 = 0.0036$
  P = 0.1 \times 0.1 \times 0.6 \times 0.6 = 0.0036

Step-3 Normalize the probability

Segmentation	Raw Probability	Normalized Probability
`[aa] [abc] [abc]`	0.108	$\frac{0.108}{0.1116} \approx 0.968$ \frac{0.108}{0.1116} \approx 0.968
`[a, a] [abc] [abc]`	0.0036	$\frac{0.0036}{0.1116} \approx 0.032$ \frac{0.0036}{0.1116} \approx 0.032

Step-4 Calculate expected count for each subword

Subword	Count in Segmentation 1 (`[aa] [abc] [abc]`)	Count in Segmentation 2 (`[a, a] [abc] [abc]`)	Expected Count
`a`	0	2	0·0.968 + 2·0.032 = **0.064**
`aa`	1	0	1·0.968 + 0·0.032 = **0.968**
`abc`	2	2	2·0.968 + 2·0.032 = **2.000**

Step-4 Calculate expected count for each subword

Subword	Expected Count	Updated Probability	Calculation Breakdown
`a`	0.064	**0.0211**	0.064 ÷ 3.032 = 0.0211
`aa`	0.968	**0.3193**	0.968 ÷ 3.032 = 0.3193
`abc`	2.000	**0.6596**	2.000 ÷ 3.032 = 0.6596

Conclusions

As seen the probability of tokens aa and abc increased and a decreased because the EM step saw that in the training corpus aa abc abc

- ‘a’ did not occur
- ‘aa’ occurred once so it increased slightly
- ‘abc’ occurred twice so it increased more

Final Tokenization of aa abc abc using Unigram

- Given the initial learned probabilities and the subwords, Unigram model finds the most probable segmentation using the viterbi algorithm then using the EM algorithm it updates their learned probabilities. The most probable segmentation then is chosen as the final tokenized corpus.
- So for the given vocabulary and corpus the best tokenization of “aa abc abc” is

“aa abc abc”

Embedding

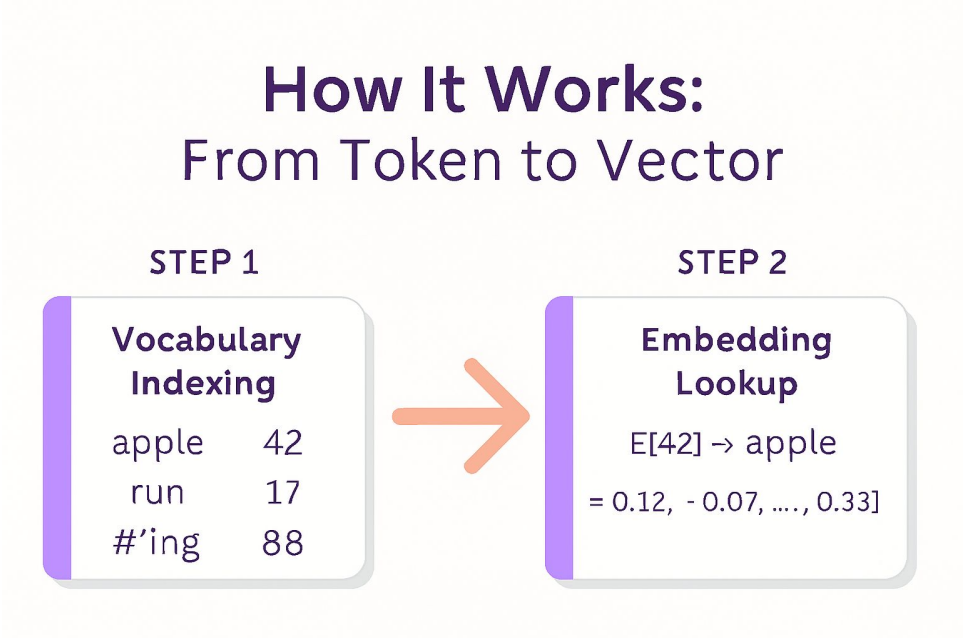
- In the starting we discussed that for a LLM model to understand process and generate language, the language must be modelled and represented as numericals. Tokenization solves the first part and models language as tokens.

Token embeddings are **dense vector representations** of tokens (words, subwords, or characters) that capture their **semantic and syntactic properties** in a continuous space.

Instead of treating tokens as discrete symbols (like one-hot vectors), embeddings allow models to **learn relationships** between tokens based on context and usage.

What Is `E[42]`?

- `E` is the **embedding matrix**: a learnable table of shape V×dV \times d, where:
  - VV = vocabulary size (e.g., 30,000 tokens)
  - dd = embedding dimension (e.g., 300 or 768)
- `E[42]` means: → Take the **42nd row** of this matrix → That row is the **embedding vector** for the token `"apple"`



Embedding Vector

- An **embedding vector** is a dense, numerical representation of a token (word, subword, or character).
- Instead of using sparse one-hot encodings, embeddings map tokens to **continuous vector spaces**.
- Each token is assigned a vector of fixed dimension (e.g., 300 or 768), learned during training.

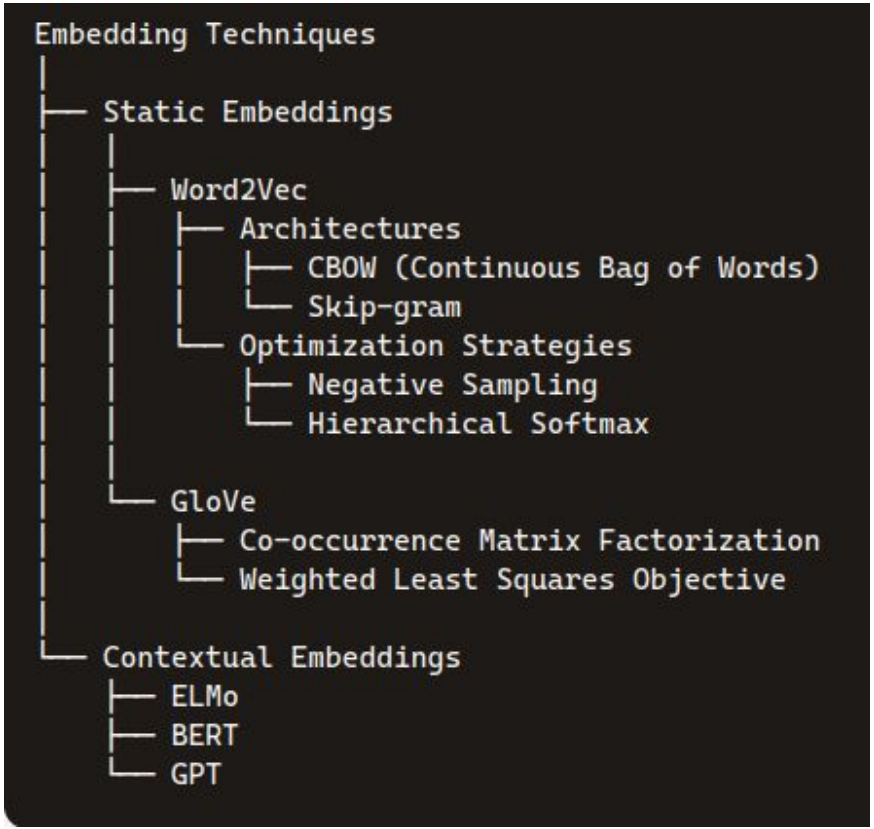
How Do Embeddings Help in Context?

- In **contextual models** (like BERT or GPT), embeddings are **dynamic**:
  - `"bank"` in `"river bank"` ≠ `"bank"` in `"money bank"`
- The model adjusts embeddings based on surrounding words, enabling:
  - **Disambiguation**
  - **Context-aware predictions**
  - **Better downstream performance** (classification, translation, etc.)

📊 Example Embedding Matrix (dim = 3)

Token	Embedding Vector
`king`	[0.52, 0.81, 0.30]
`queen`	[0.50, 0.79, 0.28]
`man`	[0.49, 0.75, 0.25]
`woman`	[0.47, 0.73, 0.23]

Types of Embedding



Static Embedding - Word2Vec

For each word, we consider **2 words before and 2 words after** as its context. Let's few examples:

🔹 **Central Word:** `make`

Context Window: `shalt`, `not`, `a`, `machine`

- **CBOW**:
  - Input: `[shalt, not, a, machine]`
  - Target: `make`
- **Skip-gram**:
  - Input: `make`
  - Targets: `shalt`, `not`, `a`, `machine`

🔹 **Central Word:** `likeness`

Context Window: `the`, `in`, `of`, `a`

- **CBOW**:
  - Input: `[the, in, of, a]`
  - Target: `likeness`
- **Skip-gram**:
  - Input: `likeness`
  - Targets: `the`, `in`, `of`, `a`

We move a window across the sentence, focusing on one **center word** at a time and collecting **2 words before and 2 words after** as context.

🧠 What's Happening?

- **CBOW**: Predict the center word from its context.
  - Learns: "If I see `shalt`, `not`, `a`, `machine`, the center is likely `make`."
- **Skip-gram**: Predict context words from the center word.
  - Learns: "If I see `make`, it's likely surrounded by `shalt`, `not`, `a`, `machine`."

Static Embedding - Word2Vec - Negative Sampling

If, however, we have a dataset of only a target value of 1, then a model can cheat and ace it by outputting 1 all the time. To get around this, we need to enrich our training dataset with examples of words that are not typically neighbors. These are called negative examples and are shown in Figure 2-13.

Word 1	Word 2	Target	
not	thou	1	Positive examples
not	shalt	1	
not	make	1	
not	a	1	
thou	apothecary	0	Negative examples
not	sublime	0	
make	def	0	
a	playback	0	

Figure 2-13. We need to present our models with negative examples: words that are not usually neighbors. A better model is able to better distinguish between the positive and negative examples.

Contextual Embeddings

Unlike traditional embeddings (Word2Vec, GloVe), which assign **one static vector per word**, contextual embeddings generate **different vectors for the same word depending on its sentence context**.

Example:

- `"bank"` in `"river bank"` → vector related to nature
- `"bank"` in `"money bank"` → vector related to finance

Same word, **different meaning**, and therefore **different embedding**.

What Are BERT Embeddings?

In BERT, each input token is transformed into a **contextual embedding**—a dense vector that reflects not just the token itself, but its meaning within the sentence.

Unlike Word2Vec or GloVe, BERT embeddings are **dynamic**:

The same word will have different embeddings depending on its context.